ViandasSemana

Módulo 2 — Menús

Menú del día con cupo, precio y horario de corte

Segundo Enriquez Gatti · Roman Oberlander
6.° 1.ª — E.T. N.° 21 — Programación sobre Redes — 2026

Módulo 2 — Menús

Este módulo permite que quien cocina publique el menú de cada día y que los clientes vean qué hay para comer, cuánto cuesta y cuántas viandas quedan.

El menú del día

Cada día hábil tiene **un único menú** con su descripción (por ejemplo "Milanesa con puré"), su **cupo máximo** de viandas, su **precio** en pesos y su **horario de corte**: la hora hasta la que se puede reservar o cancelar ese día. Después de esa hora el menú se muestra "Cerrado".

El cupo existe porque la cocina tiene una capacidad fija de producción: si se venden más viandas de las que se pueden cocinar, alguien se queda sin comer. Por eso el sistema nunca acepta una reserva que lo supere.

Lo que ve cada rol

Rol	Qué ve en "Menú de la semana"	Qué puede hacer
CLIENTE	Una tarjeta por día con descripción, precio, viandas reservadas sobre el cupo, la etiqueta "Quedan N", "Agotado" o "Cerrado" y la hora de corte.	Elegir la cantidad y tocar "Reservar y pagar" (módulo 3).
COCINERO	Las mismas tarjetas, con cuántas viandas ya se reservaron.	Crear, editar y borrar menús.

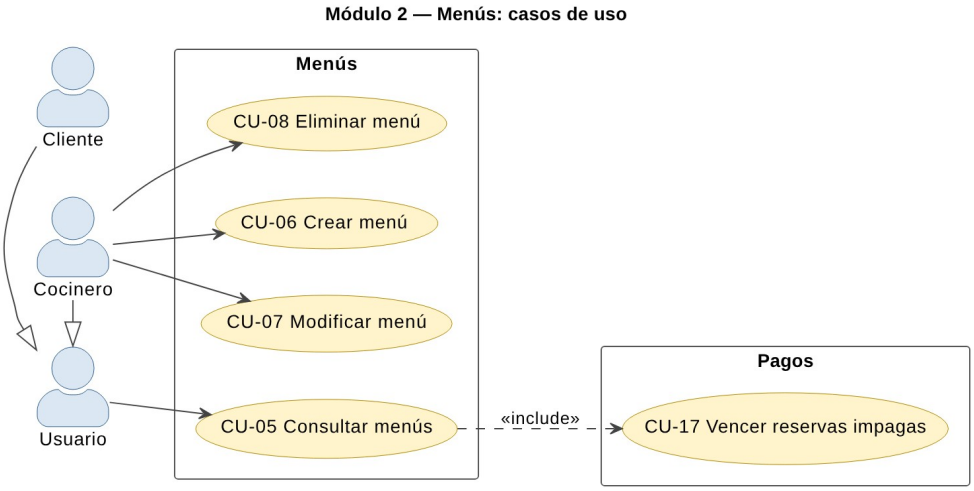
Disponibilidad calculada por el servidor

El frontend no calcula ninguna regla. La API devuelve en cada menú tres datos ya calculados: **reservadas** (viandas de reservas pendientes de pago y confirmadas), **disponibles** (cupo menos reservadas, nunca negativo) y **abierto** (si tiene precio y todavía no pasó la hora de corte). La pantalla sólo los muestra.

Antes de calcular, el sistema cancela las reservas que no se pagaron a tiempo, para que el cupo que se muestra sea el real.

Los días pasados se ocultan por defecto; el interruptor "Ver días anteriores" los muestra.

Diagrama de casos de uso



Casos de uso del módulo de menús (archivo editable: diagramas/cu-2-menus.drawio).

Casos de uso

CU-05 Consultar menús

CU-05	Consultar menús		
Descripción	Un usuario con sesión iniciada ve los menús de la semana, cada uno con su precio, su hora de corte y cuántas viandas quedan. El cliente ve si todavía puede reservar; el cocinero, cuántas viandas ya se reservaron.		
Actores	Usuario (Cliente o Cocinero).		
Precondiciones	El usuario tiene la sesión iniciada (CU-02).		
Postcondiciones	No cambia ningún menú. Las reservas impagas que ya vencieron quedan canceladas y dejan de ocupar cupo.		
Secuencia normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Abre la pestaña "Menú de la semana".	Pide GET /api/menus con el token.
	2		Valida la sesión.
		2.1 Si no hay sesión o el token venció, el sistema deberá responder 401 y volver a la pantalla de ingreso.	
	3		Cancela las reservas cuyo plazo para pagar ya venció (realiza el caso de uso CU-17).
	4		Lee los menús ordenados por fecha y suma, para cada uno, las viandas de los pedidos PENDIENTE_PAGO y CONFIRMADO.
	5		Devuelve cada menú con reservadas, disponibles (cupo − reservadas, nunca negativo) y abierto (tiene precio y no pasó la hora de corte).
	6	Ve una tarjeta por día.	Muestra precio, barra de cupo y hora de corte. Al cliente le agrega "Reservar y pagar" si el menú está abierto y quedan viandas; al cocinero, "Editar" y "Borrar".
		6.1 Si el usuario activa "Ver días anteriores", el sistema deberá mostrar también los menús de días pasados (por defecto se ocultan).	
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	1		En el caso de que se pida un menú por id que no existe (GET /api/menus/:id), el sistema deberá responder 404 "Menú no encontrado".

	2		En el caso de que la base de datos no responda, el sistema deberá responder 500 sin detalles internos y registrar error_servidor.
Rendimiento	El sistema deberá realizar las acciones de los pasos 2 al 5 en un máximo de 1 segundo.		
Frecuencia	Este caso de uso se espera que se lleve a cabo una media de 60 veces al día (es la pantalla principal).		
Importancia	Vital.		
Urgencia	Inmediatamente.		
Comentarios	La API calcula reservadas, disponibles y abierto; el frontend sólo los muestra (consigna 3.2).		

CU-06 Crear menú

CU-06	Crear menú		
Descripción	El cocinero publica el menú de un día con su descripción, cupo máximo, precio y hora de corte para que los clientes puedan reservarlo.		
Actores	Cocinero.		
Precondiciones	El cocinero tiene la sesión iniciada con rol COCINERO.		
Postcondiciones	Existe un MenuDia nuevo para esa fecha y los clientes lo ven abierto hasta la hora de corte.		
Secuencia normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Toca "Nuevo menú".	Muestra el formulario con fecha, descripción, cupo máximo, precio y hora de corte.
	2	Completa los datos y guarda.	Envía POST /api/menus y verifica que el rol sea COCINERO.
		2.1 Si el usuario es CLIENTE, el sistema deberá responder 403 "No tenés permiso para esta acción".	
	3		Valida los datos: fecha AAAA-MM-DD, descripción no vacía, cupo entero ≥ 1, precio entero ≥ 1 (pesos) y hora HH:mm entre 00:00 y 23:59.
		3.1 Si algún dato es inválido, el sistema deberá responder 400 con el campo y el mensaje (por ejemplo "Formato HH:mm").	
	4		Verifica que no haya otro menú para esa fecha.
		4.1 Si ya existe un menú para esa fecha, el sistema deberá responder 409 y no crear nada.	
	5		Guarda el menú y responde 201 con sus totales (0 reservadas).
	6	Ve el menú nuevo en la lista.	Muestra "Menú creado".
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	1		En el caso de que la base de datos no responda, el sistema deberá responder 500 y registrar error_servidor.
Rendimiento	El sistema deberá realizar las acciones de los pasos 2 al 5 en un máximo de 1 segundo.		
Frecuencia	Este caso de uso se espera que se lleve a cabo una media de 5 veces por semana (un menú por día hábil).		
Importancia	Vital.		
Urgencia	Inmediatamente.		
Comentarios	La hora de corte se interpreta como hora de Argentina (UTC-3). El precio va en pesos enteros, sin centavos.		

CU-07 Modificar menú

CU-07	Modificar menú		
Descripción	El cocinero cambia los datos de un menú ya publicado (descripción, fecha, cupo, precio u hora de corte).		
Actores	Cocinero.		
Precondiciones	El cocinero tiene la sesión iniciada con rol COCINERO y el menú existe.		
Postcondiciones	El menú queda con los datos nuevos y sus totales recalculados. Las reservas existentes se mantienen.		
Secuencia normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Toca "Editar" en un menú.	Abre el formulario con los datos actuales.
	2	Cambia los datos y guarda.	Envía PUT /api/menus/:id, verifica el rol y valida los datos igual que en CU-06.
		2.1 Si el usuario es CLIENTE, el sistema deberá responder 403; si los datos son inválidos, 400.	
	3		Verifica que el menú exista y que la fecha nueva no la tenga otro menú.
		3.1 Si el menú no existe, el sistema deberá responder 404; si la fecha ya la usa otro menú, 409.	
	4		Actualiza el menú y lo devuelve con reservadas, disponibles y abierto recalculados.
		4.1 Si el cupo nuevo queda por debajo de lo ya reservado, el sistema deberá mostrar 0 disponibles (nunca un número negativo) y no aceptar más pedidos, sin cancelar las reservas que ya existen.	
	5	Ve el menú actualizado.	Muestra "Menú actualizado".
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	1		En el caso de que la base de datos no responda, el sistema deberá responder 500 y registrar error_servidor.
Rendimiento	El sistema deberá realizar las acciones de los pasos 2 al 4 en un máximo de 1 segundo.		
Frecuencia	Este caso de uso se espera que se lleve a cabo una media de 2 veces por semana.		
Importancia	Importante.		
Urgencia	Hay presión.		
Comentarios	Permite corregir errores de carga sin borrar el menú, lo que sería imposible si ya tiene pedidos.		

CU-08 Eliminar menú

CU-08	Eliminar menú		
Descripción	El cocinero borra un menú que todavía no tiene pedidos, por ejemplo uno cargado por error.		
Actores	Cocinero.		
Precondiciones	El cocinero tiene la sesión iniciada con rol COCINERO; el menú existe y no tiene pedidos.		
Postcondiciones	El menú deja de existir y desaparece de la lista de todos los usuarios.		
Secuencia normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Toca "Borrar" en un menú.	Pide confirmación: "¿Borrar el menú del …?".
		1.1 Si el cocinero no confirma, el sistema deberá no enviar nada y dejar el menú como estaba.	
	2	Confirma.	Envía DELETE /api/menus/:id y verifica el rol y que el menú exista.
		2.1 Si el usuario es CLIENTE, el sistema deberá responder 403; si el menú no existe, 404.	
	3		Cuenta los pedidos del menú, de cualquier estado.
		3.1 Si el menú tiene pedidos, el sistema deberá responder 409 "No se puede borrar un menú que tiene pedidos" y no borrarlo.	
	4		Borra el menú y responde 204.
	5	Ve la lista sin ese menú.	Recarga la lista de menús.
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	1		En el caso de que se intente borrar el menú directamente en la base teniendo pedidos, la clave foránea (ON DELETE RESTRICT) deberá impedirlo.
Rendimiento	El sistema deberá realizar las acciones de los pasos 2 al 4 en un máximo de 1 segundo.		
Frecuencia	Este caso de uso se espera que se lleve a cabo una media de 1 vez por mes.		
Importancia	Quedaría bien.		
Urgencia	Puede esperar.		
Comentarios	Un menú con pedidos no se borra para no perder el historial de reservas y pagos.		

Funcionamiento técnico

1. Arquitectura y stack tecnológico

- **API:** Express 5 + TypeScript, router `menusRouter` en `backend/src/routes/menus.ts`.
- **Validación:** esquema zod `esquemaMenu` (fecha, descripción, cupo, precio y hora de corte).
- **Persistencia:** PostgreSQL con Prisma, tabla `MenuDia`. La suma de viandas reservadas de cada menú se hace con `prisma.pedido.aggregate` sobre `Pedido`.
- **Reglas puras:** `yaPasoElCorte` e `instanteDeCorte` en `backend/src/domain/reglasPedido.ts` (se prueban sin base de datos).
- **Frontend:** React 18 + Vite, pantalla `frontend/src/pantallas/Menus.tsx`.

2. Permisos

Todas las rutas del módulo exigen sesión (`menusRouter.use(autenticar)`). Las de escritura exigen además el rol COCINERO:

```
menusRouter.get('/', ...)                                  // cualquier usuario con sesión
menusRouter.post('/', exigirRol('COCINERO'), ...)          // 403 para un CLIENTE
menusRouter.put('/:id', exigirRol('COCINERO'), ...)
menusRouter.delete('/:id', exigirRol('COCINERO'), ...)
```

3. Cálculo de disponibilidad (conTotales)

1. Se ejecuta `vencerReservasImpagas()` (CU-17) para liberar el cupo de las reservas vencidas.
2. Para cada menú se suma la `cantidad` de sus pedidos (`aggregate`), sólo de los estados que **ocupan cupo**: `PENDIENTE_PAGO` y `CONFIRMADO` (constante `OCUPAN_CUPO`).
3. `disponibles = max(0, cupoMaximo − reservadas)`.
4. `abierto = precio > 0 && !yaPasoElCorte(fecha, horaCorte, ahora)`.

Hora de corte

`horaCorte` se guarda como texto "HH:mm" y se interpreta en **hora de Argentina (UTC-3, sin horario de verano)**. `instanteDeCorte` la convierte al instante UTC exacto; así el resultado es el mismo en la computadora de desarrollo y en el servidor, que corre en UTC.

4. Esquema en la base de datos

Campo	Tipo	Notas
id	entero, PK	autoincremental
fecha	DATE, **único**	un menú por día (índice MenuDia_fecha_key)
descripcion	texto	no vacía
cupoMaximo	entero	≥ 1
horaCorte	texto "HH:mm"	hora de Argentina
precio	entero	pesos; 0 sólo en menús anteriores a los pagos (no se pueden reservar)

Relación: un `MenuDia` tiene muchos `Pedido`. La clave foránea es `ON DELETE RESTRICT`.

5. Endpoints

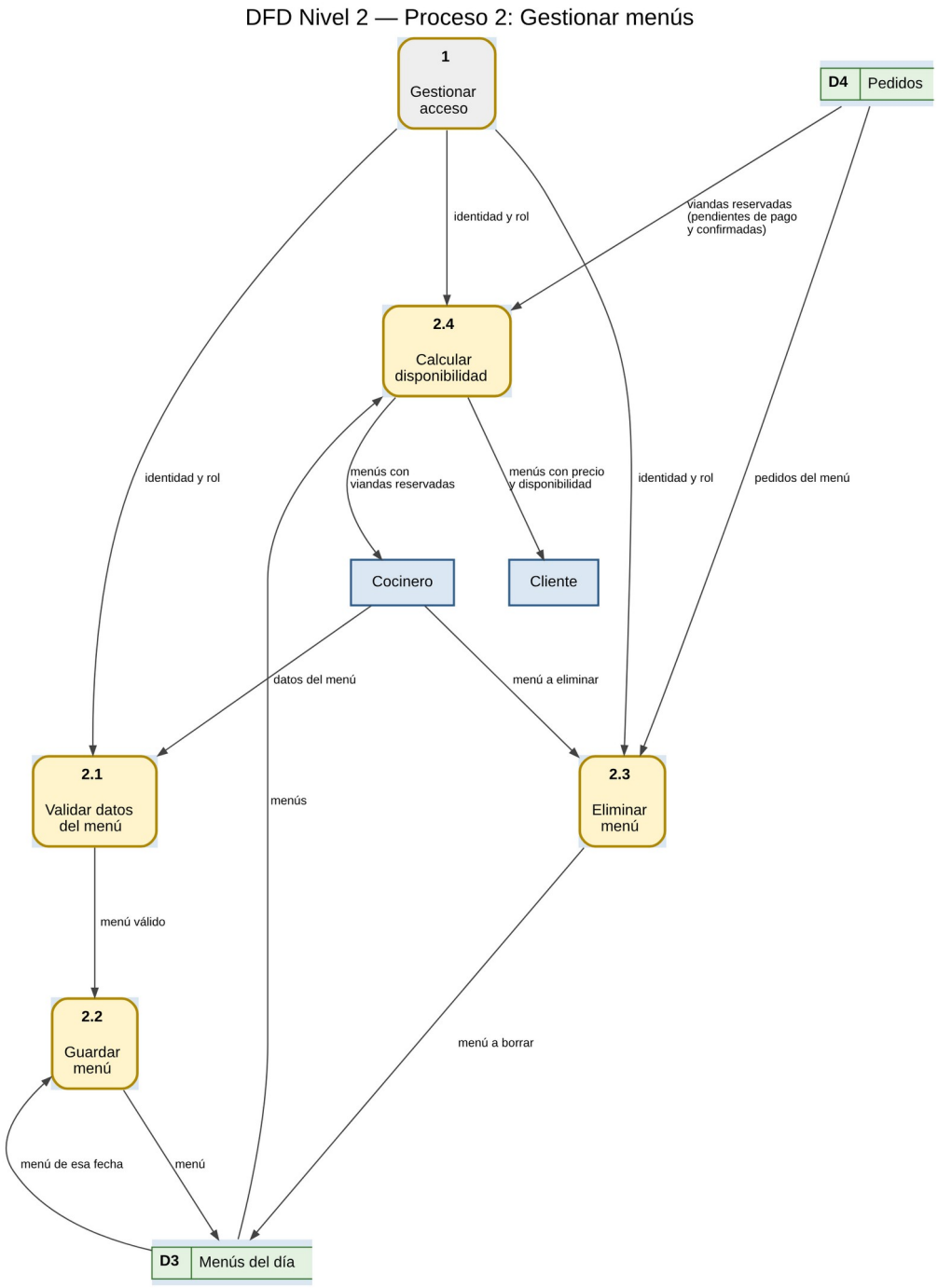
Método y ruta	Rol	Respuestas
GET /api/menus	cualquiera con sesión	200, 401
GET /api/menus/:id	cualquiera con sesión	200, 400, 401, 404
POST /api/menus	COCINERO	201, 400, 401, 403, 409
PUT /api/menus/:id	COCINERO	200, 400, 401, 403, 404, 409
DELETE /api/menus/:id	COCINERO	204, 400, 401, 403, 404, 409

El cocinero puede agregar una **foto opcional del plato** al crear o editar un menú. El navegador la achica a 800 px y la pasa a JPEG; la API acepta JPG, PNG o WebP de hasta 300.000 caracteres y la guarda en `MenuDia.imagen`. La lista de pedidos no devuelve la foto, para no pesar de más.

Ejemplo de respuesta de `GET /api/menus`:

```
[{ "id": 3, "fecha": "2026-10-15T00:00:00.000Z", "descripcion": "Milanesa con puré",
   "cupoMaximo": 20, "horaCorte": "10:00", "precio": 6500,
   "reservadas": 12, "disponibles": 8, "abierto": true }]
```

6. DFD nivel 2 — proceso 2 "Gestionar menús"



Archivo editable: diagramas/dfd-2-menus.drawio.

N.º	Proceso	Entrada	Salida	Almacenes
2.1	Validar datos del menú	datos del menú, identidad y rol	menú válido	—
2.2	Guardar menú	menú válido	—	D3 (lee la fecha, escribe el menú)
2.3	Eliminar menú	menú a eliminar, identidad y rol	—	D4 (lee sus pedidos), D3 (borra)
2.4	Calcular disponibilidad	identidad y rol	menús con precio y disponibilidad	D3 (lee), D4 (suma viandas)

7. Reglas y dónde se validan

Regla	Dónde
Formato de fecha, hora de corte, cupo y precio	`esquemaMenu` (zod) en routes/menus.ts
Un menú por fecha	índice único + control previo (409) en routes/menus.ts
Un menú sin precio no se puede reservar	`abierto` en conTotales y control SIN_PRECIO en routes/pedidos.ts
No se borra un menú con pedidos	routes/menus.ts (409) y FK ON DELETE RESTRICT

8. Archivos del código

Archivo	Contenido
backend/src/routes/menus.ts	CRUD de menús y conTotales.
backend/src/domain/reglasPedido.ts	yaPasoElCorte, instanteDeCorte.
backend/src/lib/pagos.ts	OCUPAN_CUPO y vencerReservasImpagas.
frontend/src/pantallas/Menus.tsx	Tarjetas, formulario del cocinero y botón "Reservar y pagar".

9. Tests que lo cubren

- **menus.test.ts (12):** guardar, conservar y quitar la foto del plato, rechazar una imagen inválida o muy pesada (400), crear (201), listar y ver uno, modificar, eliminar, no borrar con pedidos (409), no repetir fecha (409), datos inválidos (400), menú inexistente (404), sin token (401) y cliente que intenta crear (403).
- **reglasPedido.test.ts:** conversión de la hora de corte a UTC y casos un minuto antes y justo en el corte.